

STACK OPERATIONS AND AMORTIZED COST ANALYSIS

Aim

To implement stack operations using a dynamically resizing array and calculate the amortized cost of push operations.

Algorithm

1. Initialize stack with size 2, top = -1, cost = 0 and pushes = 0.
2. Read the number of push operations.
3. If the stack is full, create a new array of double size and copy elements.
4. Insert the element and update cost and push count.
5. Perform pop operations as required.
6. Calculate and display the amortized cost.

Program

```
import java.util.Scanner;

public class StackAmortized {
    static int[] a = new int[2];
    static int top = -1;
    static long cost = 0, pushes = 0;

    static void push(int x) {
        if (top + 1 == a.length) {
            int old = a.length, n = old * 2;
            int[] b = new int[n];
            for (int i = 0; i <= top; i++) b[i] = a[i];
            a = b; cost += old;
        }
        a[++top] = x; cost++; pushes++;
    }

    static int pop() { return a[top--]; }
    static double amortized() { return pushes == 0 ? 0 : (double) cost / pushes; }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter no. of pushes: ");
        int n = sc.nextInt();
        for (int i = 1; i <= n; i++) push(i);
        System.out.print("Enter no. of pops: ");
        int p = sc.nextInt();
        for (int i = 0; i < p && top >= 0; i++) pop();
        System.out.println("Total cost = " + cost);
        System.out.println("Pushes = " + pushes);
        System.out.printf("Amortized cost = %.2f\n", amortized());
    }
}
```

Output

```
Enter no. of pushes: 12
Enter no. of pops: 0
Total cost = 26
Pushes = 12
Amortized cost = 2.17
```

Result

Thus, the program to implement stack operations and calculate the amortized cost using dynamic array resizing has been executed successfully. The total cost, number of push operations, and amortized cost were calculated and displayed correctly.